

Predictive Model for Customer Churn Analysis

Machine Learning - BeeNeural Adaptive Task

This report documents a complete telecom churn machine-learning workflow: reproducible data generation, cleaning, feature engineering, model comparison, model tuning, evaluation, business-oriented threshold selection, model persistence, and deployment.

Item	Result
Dataset	6,000 reproducible telecom customer records
Target	Churn (Yes/No)
Selected model	Logistic Regression
ROC-AUC	0.7864
PR-AUC	0.7290
Deployment threshold	0.33
Recall	0.8889
F1	0.7348

1. Problem Statement and Business Goal

Customer churn directly affects recurring revenue. The objective is to estimate the probability that a telecom customer will leave so retention teams can prioritize outreach. The model is therefore evaluated not only by accuracy but also by recall, F1, ROC-AUC and PR-AUC. In deployment, the decision threshold is tuned to catch a larger share of actual churners.

2. Data and Preprocessing

The package includes a fixed-seed synthetic telecom dataset so the submission is fully reproducible without requiring external credentials or downloads. It contains demographics, tenure, subscribed services, contract/billing information, charges and the churn target. A small number of missing TotalCharges values are intentionally introduced and repaired inside the preprocessing workflow.

3. Feature Engineering

Eleven business-oriented features are created before modeling:

- NumServices - count of active services
- HasInternet - internet subscription flag
- HasSupportBundle - both security and tech support active
- AutoPay - automatic payment indicator
- FamilyAccount - partner/dependent account flag
- AvgMonthlySpend - historical average spend
- ChargeIncreaseRatio - current vs. historical spend ratio
- TenureBand - lifecycle bucket
- HighMonthlyCharge - high-charge indicator
- MonthToMonth - short-contract risk flag
- SupportGap - internet customer lacking support/security

All transformations are embedded in a scikit-learn Pipeline/ColumnTransformer so preprocessing learned on training data is consistently applied during validation, test evaluation and deployment.

4. Model Selection and Tuning

Logistic Regression, Random Forest and Histogram Gradient Boosting are compared using stratified 3-fold cross-validation. A compact validation-based Random Forest tuning step evaluates multiple tree-depth, size and feature-sampling configurations. The final model is chosen using held-out ROC-AUC.

Model	CV ROC-AUC	CV F1	Test ROC-AUC
Logistic Regression	0.7753	0.6961	0.7864
Random Forest	0.7652	0.6877	0.7795
Hist Gradient Boosting	0.7556	0.6652	0.7746

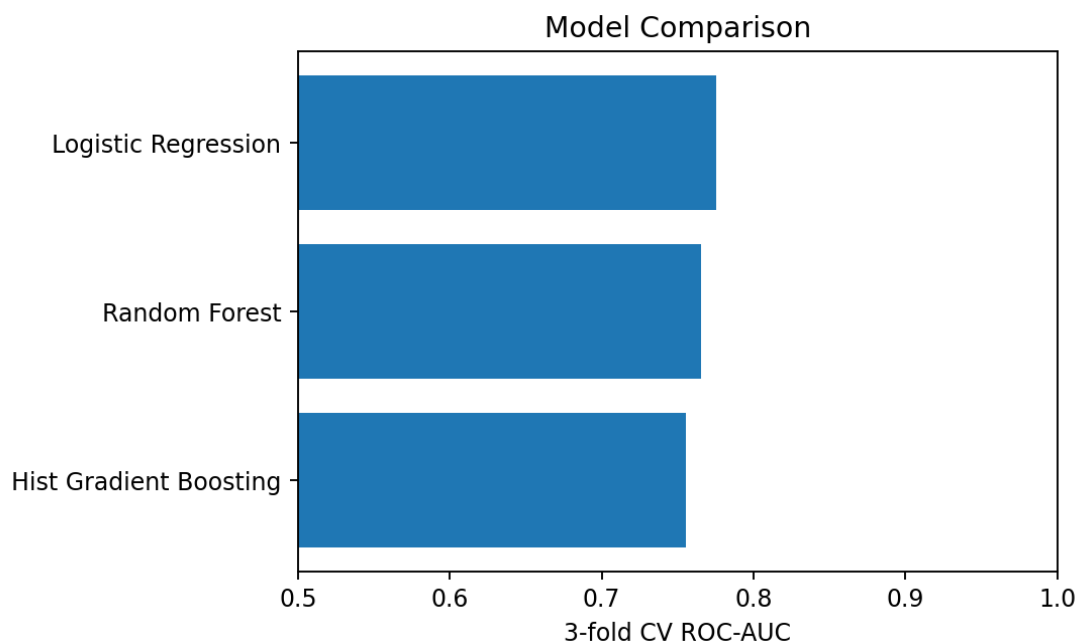


Figure 1. Model comparison by cross-validated ROC-AUC.

5. Final Evaluation

At the selected threshold of 0.33, the model achieves accuracy 0.7017, precision 0.6263, recall 0.8889, F1 0.7348, ROC-AUC 0.7864, and PR-AUC 0.7290. The threshold is deliberately lower than 0.50 to raise recall for churners, reflecting a retention use case where missing a true churner can be costly.

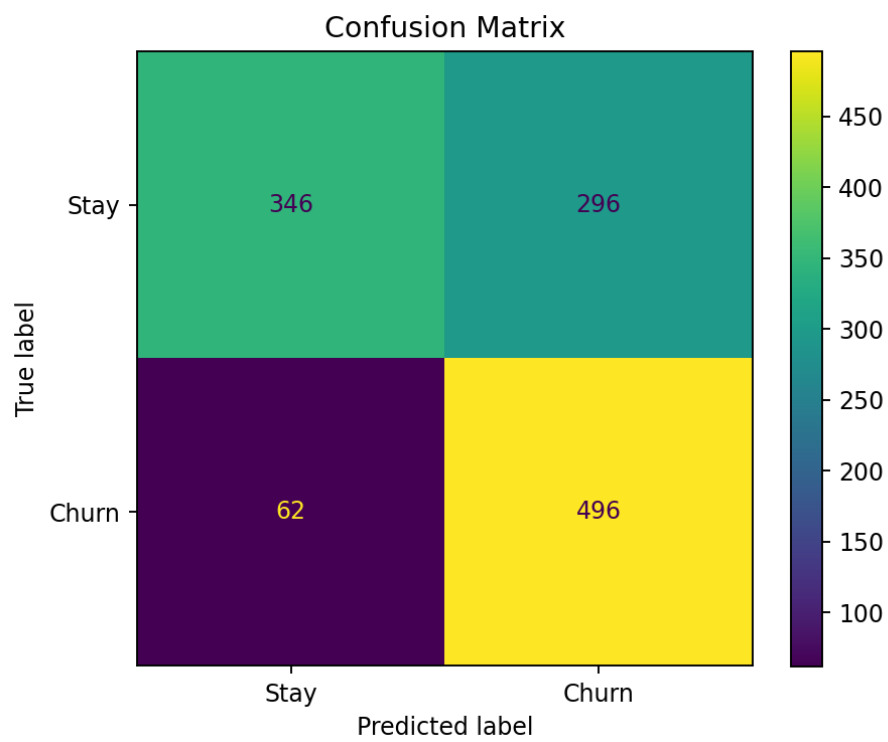


Figure 2. Confusion matrix at the deployment threshold.

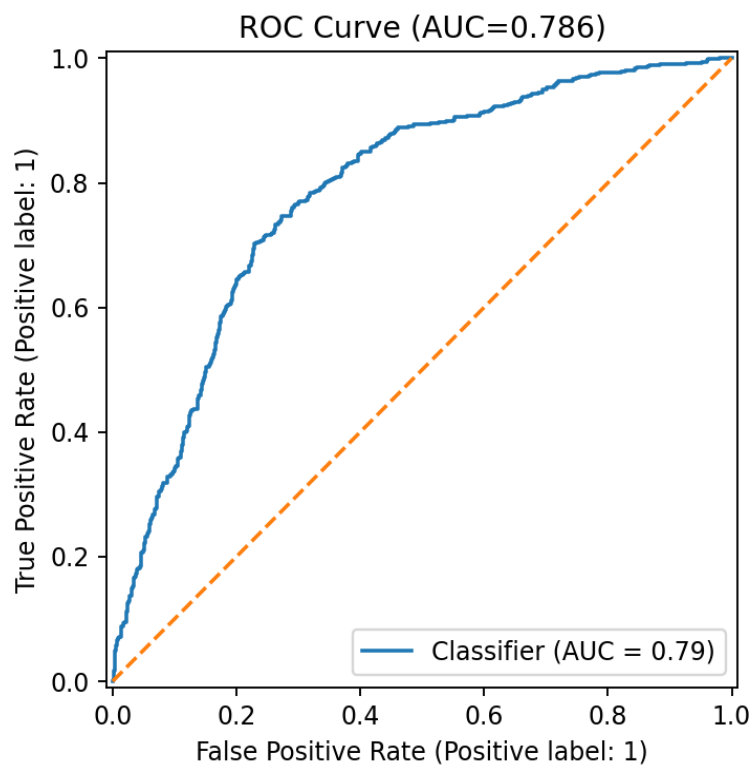


Figure 3. ROC curve.

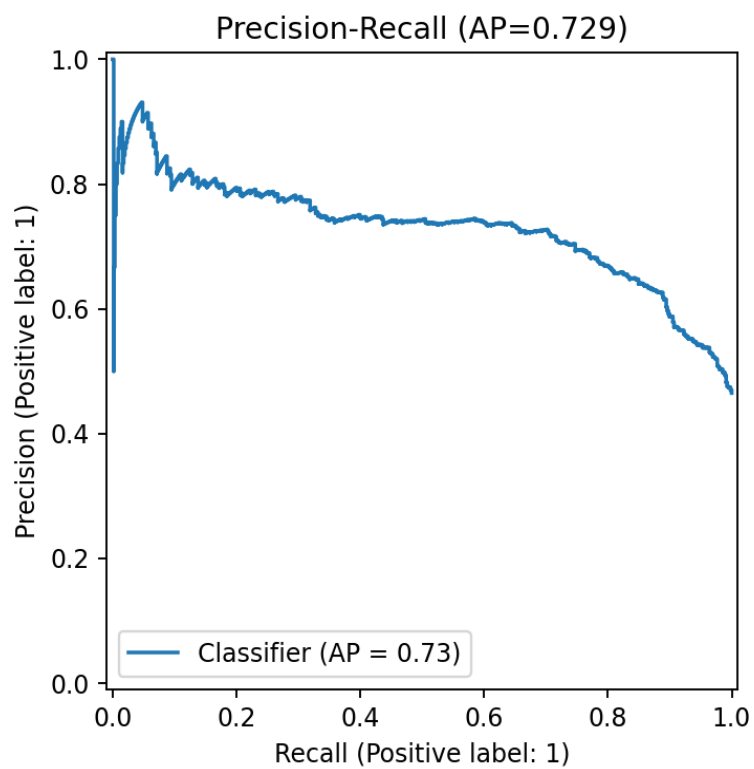


Figure 4. Precision-recall curve.

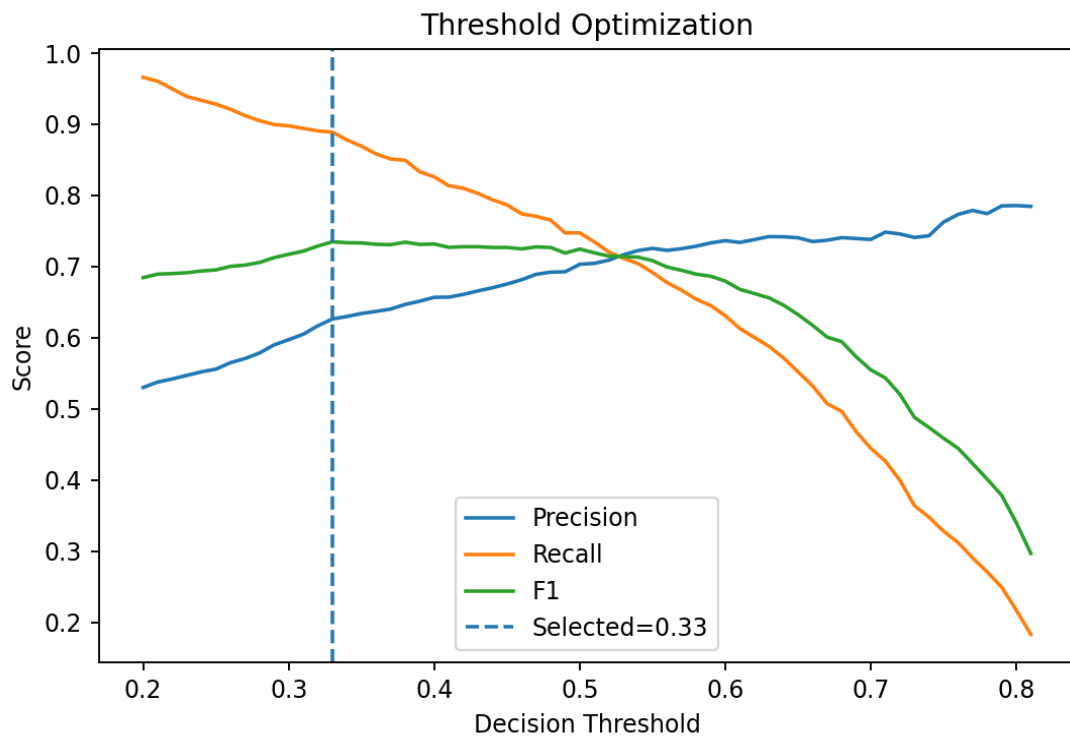


Figure 5. Precision, recall and F1 across decision thresholds.

6. Feature Importance and Interpretation

Permutation importance measures the decrease in ROC-AUC when each original feature is shuffled. The strongest signals in this run include month-to-month status, tenure, internet service, contract type, total charges and support-related variables. These are useful because they can map to retention actions rather than acting only as abstract predictors.

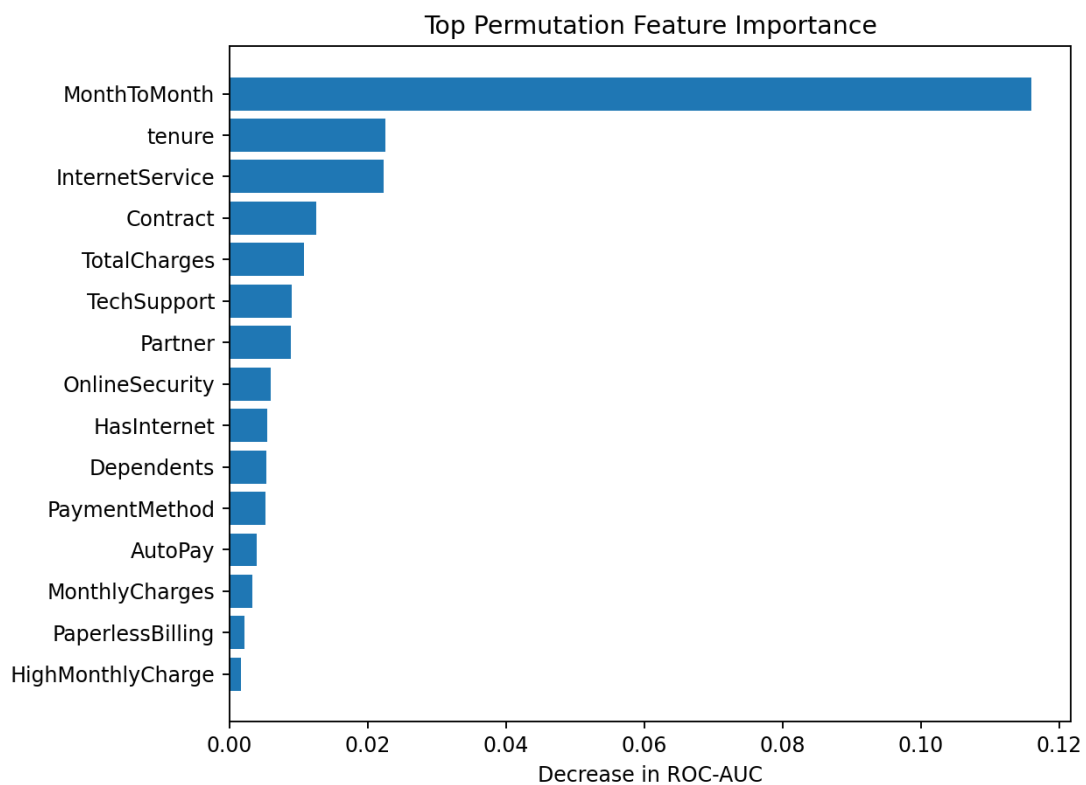


Figure 6. Top permutation feature importance.

7. Deployment

The trained sklearn pipeline is saved as `models/churn_model.joblib`. A metadata file stores the selected decision threshold. The project supports three deployment/test paths: (1) `predict.py` for command-line inference, (2) `app.py` for an interactive Streamlit web application, and (3) `Dockerfile/Procfile` files for hosted deployment. The deployment code loads the exact trained pipeline and applies the same feature-engineering function used in training.

8. Reproducibility and Deliverables

The repository includes the dataset generator, generated CSV, notebook, training code, feature-engineering module, trained model, evaluation metrics, figures, screenshots, README, requirements, Docker configuration and this PDF report. Seed 42 is used throughout where randomness is involved.

9. Limitations and Production Considerations

This academic package uses a synthetic benchmark so it is fully portable. Before production use, the same workflow should be retrained on real historical telecom churn labels and validated for population drift, calibration, fairness, privacy, operational cost, and changing customer behavior.

10. Rubric Coverage

Rubric	Evidence
Model Accuracy (30%)	Three-model comparison, stratified validation, tuning, held-out evaluation, and a saved best pipeline.
Feature Engineering (20%)	11 business features plus leakage-safe preprocessing.
Model Evaluation (25%)	Accuracy, precision, recall, F1, ROC-AUC, PR-AUC, confusion matrix, ROC/PR curves, threshold analysis, and feature importance.
Deployment (25%)	Streamlit application, CLI inference, model persistence, Dockerfile, and Procfile.